

Bipedal Robotic Locomotion with Reinforcement Learning

Drew Burcher
drewb210@vt.edu

Dan Torri
danieltorri@vt.edu

Nathan Wyatt
nwyatt04@vt.edu

ABSTRACT

We train a 13 degree of freedom bipedal humanoid (Booster Robotics T1) to walk forward in a PyBullet simulation using two model free reinforcement learning algorithms: Soft Actor-Critic (SAC) and Proximal Policy Optimization (PPO). The policy is a two-layer MLP that maps joint angle and velocity observations to joint position targets. A reward function was engineered through many trials to combine forward velocity, an alive bonus, an energy cost, and several stability penalties. After roughly 2.8 M environment steps the SAC policy reaches a mean evaluation return of about 7,100 and a forward speed of ~ 1.6 m/s, while PPO under several reward variants plateaus near a return of 3,000 and never produces a sustained gait. We discuss the reward shaping decisions that drove these results, the forgetting failures we saw with SAC late in training, and the practical trade offs between on and off policy methods on a dense reward locomotion task.

1 INTRODUCTION

Getting a bipedal robot to walk is a classic test bed for control and reinforcement learning because the system is high-dimensional, underactuated, and unstable in its natural rest state. Small mistakes early in an episode quickly result in a fall. Classical controllers exist for this problem (zero moment point trackers, model predictive control, quadratic programming whole body controllers), but they all require fairly accurate dynamics models and careful tuning, and the resulting gaits tend to not be robust to varying conditions and environments.

Reinforcement learning give the model more freedom, and flexibility. Instead of hard coding a controller, we specify what we want (move forward, stay upright, save energy) and use gradient descent on a policy for behaviors that maximize that reward. Modern actor-critic methods such as SAC [3] and PPO [9] have made this practical, and a number of recent papers [8, 10] have shown that policies trained entirely in simulation can transfer to physical hardware.

In this project, we work in simulation only, on the Booster Robotics T1 humanoid, and use the project as a test to compare of two of the most common RL algorithms. Our research question are:

- (1) Can either SAC or PPO learn a stable forward gait on the humanoid from scratch using a simple position and velocity state and a tuned reward?
- (2) How does SAC compare to PPO in terms of sample efficiency, training time, final performance, and stability of the learned policy?
- (3) Which reward terms actually shape the gait, and which are easy for the policy to "hack" into unwanted behaviors?

2 RELATED WORK

Most walking robot demos before machine learning was popular used either trajectory tracking controllers [4] or model predictive

whole body controllers [5]. These work well when the robot model is accurate, but it is very difficult to keep the robot stable on uneven ground or under disturbances.

Deep RL for humanoid walking is the current state of the art method. Schulman et al. introduced PPO [9]. It is a simple, easy to tune and on-policy. It trains every episode, but can tend to overfit to more recent data. MuJoCo's *Humanoid*, *Walker2d*, and *HalfCheetah*. Haarnoja et al. [3] introduced SAC, which adds an entropy term to the objective and uses an off-policy replay buffer. It is typically more sample efficient than PPO, but more complex, and compute intensive.

For training robots in simulation and using them in the real world, additional considerations are required. Siekmann et al. [10] train PPO policies in MuJoCo and deploy them on the Cassie biped, using domain randomization and a periodic reward to encourage stepping. Rudin et al. [8] train massively parallel PPO policies for the ANYmal quadruped in NVIDIA Isaac Gym, demonstrating that scaling up rollouts can matter more than algorithm choice. Peng et al. [6] showed that imitation rewards from motion capture data dramatically reduce the search problem for humanoid characters. However, these techniques are not within the scope of this project as we are only using a robot in simulation.

Our environment is built on PyBullet [1] and wrapped in a Gymnasium [2] interface, and we use the Stable-Baselines3 [7] implementations of SAC and PPO so the comparison reflects standard implementations rather than algorithmic differences hidden in our own code.

3 METHOD

3.1 Robot and simulator

The Booster Robotics T1 is a ~ 1.2 m humanoid with 23 revolute joints. We load its URDF in PyBullet and run physics at 240 Hz. The two head joints are locked, and the eight arm joints are held at zero by a stiff position controller, so the policy controls the remaining 13 joints (waist plus six per leg: hip pitch/roll/yaw, knee pitch, and ankle pitch/roll). Locking the arms keeps the action space small. We attempted to train a model with the arms unlocked but it still fell immediately even after 2 million steps of training.

The policy runs at 60 Hz, so each policy step advances four 240 Hz physics sub steps. Episodes last up to 1,000 policy steps (~ 16.7 s). An episode terminates early if the torso drops below 0.35 m, rises above 2 m, or tilts past $\sim 70^\circ$ in roll or pitch.

3.2 Observation and action spaces

The observation is a 36-dim float vector containing torso height, torso orientation, torso linear and angular velocity, and the 13 actuated joint positions and velocities (13+13). We don't include global x, y position so the policy works the same regardless of position.

The action $a \in [-1, 1]^{13}$ is a normalized joint angle target. We map it to each joint’s URDF limits and apply position control with PyBullet’s POSITION_CONTROL mode with PD gains $K_p=0.2$, $K_d=0.5$. The ankles use more damped gains ($K_p=0.1$, $K_d=1.0$) because we found that the robot was moving the ankles very quickly to move forward which was more of a glitch in the physics engine than a viable gait.

3.3 Reward function

We use a hand-tuned reward function applied at every policy step. With $\dot{x}$ the torso forward velocity, τ the joint torques, $\dot{q}$ the joint velocities, ϕ, θ the torso roll and pitch, q_i the joint angles relative to their normalized $[-1, 1]$ range, and $\dot{z}$ the torso vertical velocity:

$$\begin{aligned}
 r_t = & 3.0 \dot{x} && \text{forward velocity} \\
 & +2.0 && \text{alive bonus} \\
 & -0.00495 |\tau \cdot \dot{q}| && \text{energy} \\
 & -0.5 (\phi^2 + \theta^2) && \text{torso upright} \\
 & -0.2 \sum_i \max(|q_i| - 0.5, 0)^2 && \text{joint limits} \\
 & -0.5 \max(-\dot{z}, 0) && \text{anti-drop}
 \end{aligned}$$

A one-time penalty of -100 is added on episode termination by falling. The forward-velocity reward is intentionally linear and unbounded so the policy is always rewarded for moving faster. The joint-limit term is a quadratic ramp that only kicks in past 50% of the joint range so it does not interfere with normal motion. The anti-drop and torso upright terms are included mainly for quickly teaching the model to not fall down. Initially there was also a term for torso height, but this resulted in reward hacking that had the robot on the tips of it’s toes.

3.4 RL algorithms

We use the Stable-Baselines3 [7] implementations of SAC and PPO with a shared $[256, 256]$ MLP for actor and critic, $\gamma=0.99$ (discount rate for reward), and Adam at 3×10^{-4} learning rate. SAC uses a 1 M-transition replay buffer, batch size 256, soft target updates with $\tau=0.005$, and auto-tuned entropy with target entropy -5 . PPO uses 2,048 rollout steps, 10 SGD epochs, batch size 64, GAE $\lambda=0.95$, clip range 0.2, value coefficient 0.5, and entropy bonus 0.01. Both algorithms run on CPU. For our network sizes and batch sizes this was about $2\times$ faster than GPU in our setup, mostly because the PyBullet step is the bottleneck and that runs on CPU.

3.5 Training infrastructure

We created a training environment around Stable-Baselines3 that automatically commits the source tree to git before each run so that the code for each run is saved. We also made it so the hyperparameter data was saved to a JSON file. During training it runs a deterministic 5-episode evaluation every 5,000 steps and records a GIF of the last evaluation. We also render a live matplotlib dashboard alongside the PyBullet GUI so we could watch the robot and the training curves at the same time. Total training time for the SAC run reported here was approximately 20 hours across multiple sessions on a single laptop.

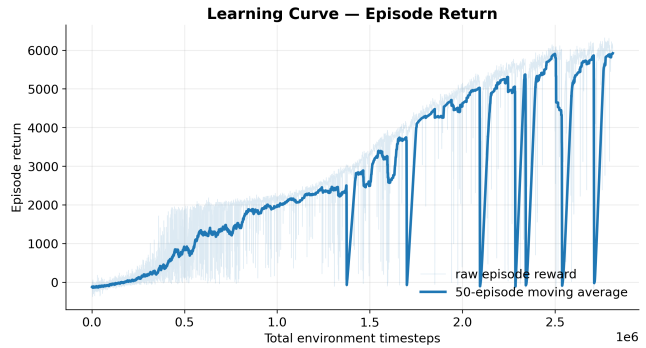


Figure 1: SAC training return per episode (light) with 50-episode moving average (dark).

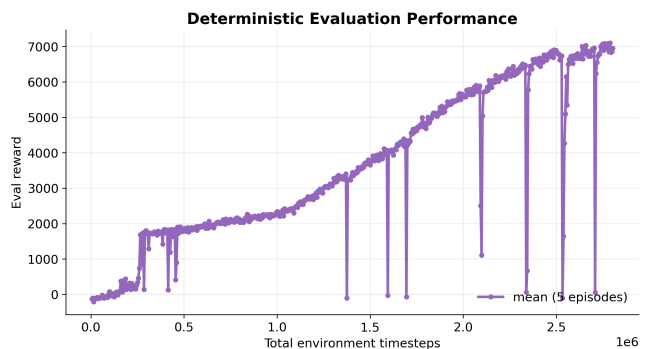


Figure 2: Deterministic evaluation return (mean over 5 episodes, action noise disabled).

4 RESULTS AND EVALUATION

We did a long SAC run (sac_legs_only_v5, 2.8 M environment steps after warm-up, accumulating across resumed sessions for a total of ~ 5.5 M trained transitions) and several shorter PPO runs spanning the same total time.

4.1 SAC: learning curves

Figure 1 shows the per episode return for SAC. The policy spends roughly 300 k steps just learning to stand, then transitions into a clear walking phase between 1.0 M and 2.0 M steps, and finally plateaus around an episode return of 6,000 with occasional collapses. The deterministic evaluation curve (Fig. 2, mean over 5 evaluation episodes with action noise disabled) is smoother and reaches $\sim 7,100$, indicating that most of the variance in the training curve comes from the stochastic policy’s exploration, not from the underlying behavior.

4.2 From standing to walking

Figure 3 shows the length of the episode (capped at 1,000) and Figure 4 shows the mean forward velocity. Together, they illustrate the two phases of learning. To keep the robot from falling for the entire episode, which the policy nails by about 1 M steps, and to actually move forward, which only ramps up after standing is solved.

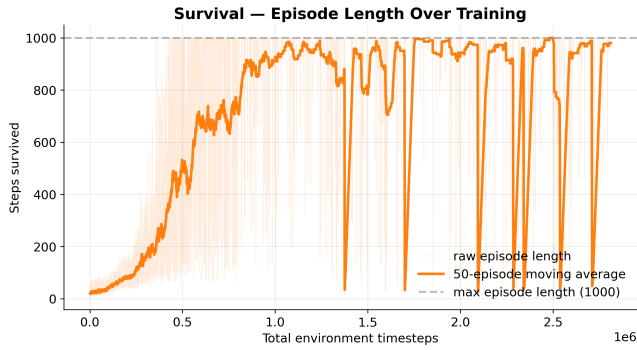


Figure 3: Episode length (steps survived). The 1,000 ceiling is reached reliably after ~ 1.0 M steps; the dips later in training match the forgetting events visible in Fig. 1.

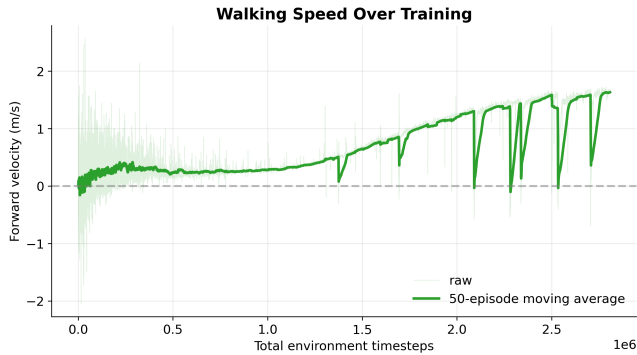


Figure 4: Mean forward torso velocity per episode. The policy is essentially stationary until 1.0 M steps and then accelerates roughly linearly with training to ~ 1.6 m/s.

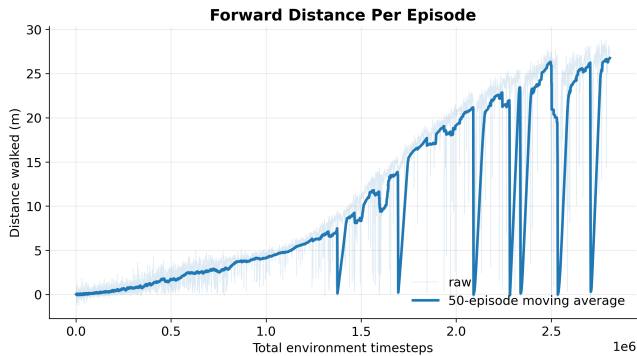


Figure 5: Forward distance walked per episode. Once the gait emerges the robot routinely covers more than 25 m in a single episode.

By 2.8 M steps the policy walks at ~ 1.6 m/s and travels ~ 26 m per 16.7 s episode (Fig. 5), which is a brisk walking pace for a robot of this scale.

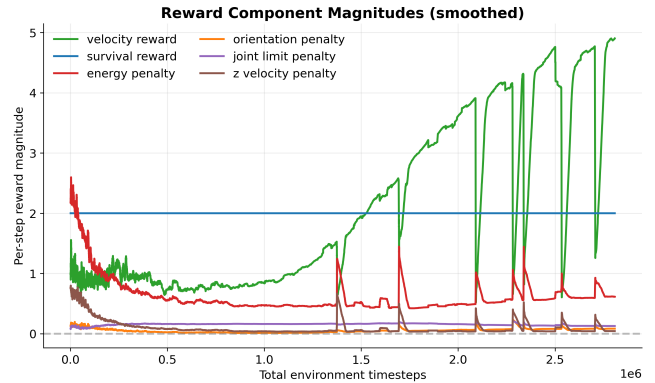


Figure 6: Mean per step magnitude of each reward term over training. Forward velocity (green) takes over once standing is solved.

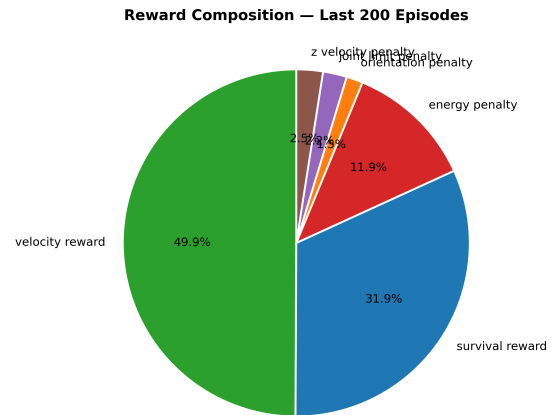


Figure 7: Reward composition over the last 200 episodes. Velocity and survival are by far the largest while the energy penalty is the next largest term.

4.3 Where the reward comes from

Figure 6 plots the mean per step magnitude of each reward term over training, and Figure 7 gives the same data as a pie chart over the last 200 episodes. Two things stand out. First, the forward velocity term grows monotonically as the policy learns to walk, while the survival bonus is by definition constant once the episode runs to the truncation step. Together they account for $\sim 82\%$ of the reward at the end of training. Second, the energy penalty is a substantial 12% of the final reward. We tried smaller energy weights and the policy converged to a behavior that moved the legs without much bend at the knee but at high frequency: it scored well on forward velocity but used roughly twice the torque per meter. The current weight balances those terms reasonably well.

Figure 8 shows the distribution of episode returns over the first 25% versus the last 25% of episodes. Early returns cluster near zero (robot falls immediately, no velocity); late returns are bimodal with

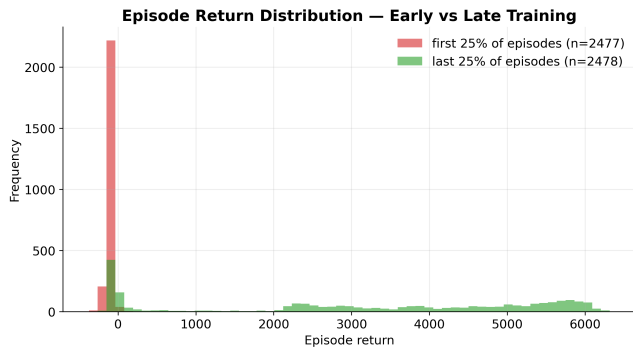


Figure 8: Histogram of per episode returns, comparing the first 25% (red) and last 25% (green) of training episodes.

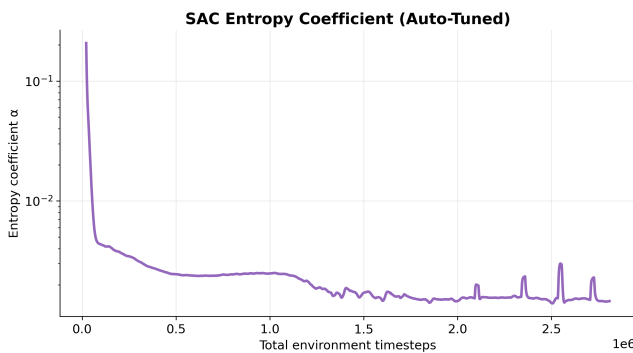


Figure 9: SAC’s auto tuned entropy coefficient α (log scale). The small bumps after 2 M steps are the controller responding to brief policy collapses.

a peak around 5,000–6,000 and a small tail near zero corresponding to the forgetting events.

4.4 Auto-tuned entropy

SAC adapts an entropy coefficient α to keep the policy near a target entropy of -5 . Figure 9 shows α on a log scale. After a brief spike during the first ~ 50 k transitions while the buffer fills, α drops two orders of magnitude and then very slowly gradually reduces further as the policy commits to a gait. The small spikes near the end of training match the forgetting dips: when the policy collapses, returns drop, the target entropy controller pushes α up to encourage re-exploration, and the policy re-learns standing from the replay buffer.

4.5 PPO progress: reward shaping for early gait formation

PPO did not converge to a fully sustained forward gait within the project timeline, but it showed clear behavioral progress across reward iterations. Early PPO policies commonly learned survival focused strategies such as standing upright, remaining mostly stationary, or making only small shuffle attempts. As reward variants were introduced to reduce those stationary solutions, PPO began

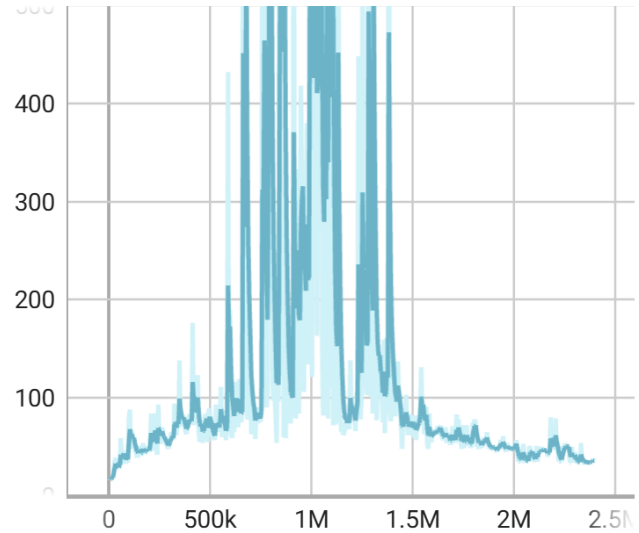


Figure 10: PPO evaluation episode length over training for the final PPO branch (gait_v23), showing improved balance and longer controlled motion in the stronger reward variants.

to expose more directional but still imperfect behaviors, including sideways drift, reverse motion “crab walk” survival, and forward fall attempts. With a stable reward scheme, these failure modes should morph into more gait-like motion over training.

Our strongest PPO result showed the beginning of a true stepping pattern where the robot started with a visible forward step with a knee bend and attempted a follow-up step before losing balance. This was not yet a sustained gait, but it was an improvement over earlier PPO behaviors and demonstrated that reward tuning moved the learned policy in the correct direction. The best PPO checkpoint achieved approximately 1.15 m of forward distance over a 1,000-step evaluation episode, corresponding to an average forward speed of roughly 0.07 m/s. This is far below the final SAC policy, but it represents the clearest PPO evidence of early gait formation.

Figure 10 shows PPO evaluation episode length over training, indicating that later reward variants improved survival and balance relative to the earliest runs. The policy fails over extended training however as seen around 1.5 million steps in. Figure 11 shows the corresponding evaluation reward. Figure 12 shows the training rollout episode length, which captures how PPO often improved early, plateaued, and then degraded under extended training. Finally, Figure ?? shows the best PPO gait attempt recorded from the final branch. Together, these results suggest that PPO was valuable less as a final locomotion solution and more as a baseline for diagnosing how reward design shaped learned behavior.

4.6 Failure modes and limitations

A few qualitative observations are worth recording, since they would shape any follow-up work:

Catastrophic forgetting. Long SAC runs are not monotonic. After the robot has been walking reliably for hundreds of thousands of steps, it can abruptly lose its balance for a few episodes and collapse the return curve to nearly zero before recovering. The dips

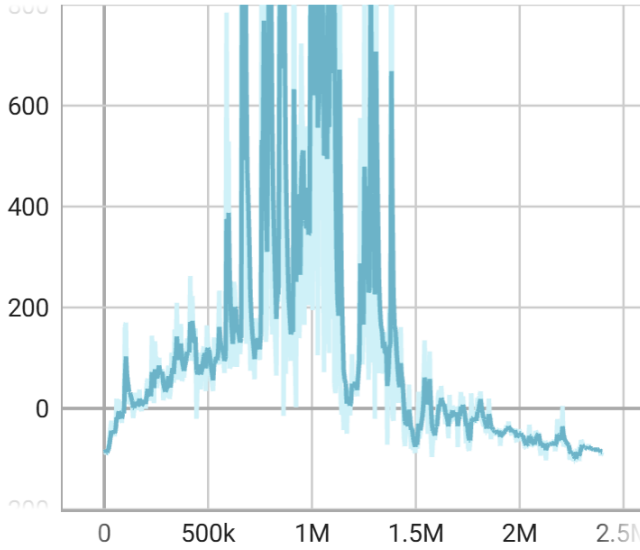


Figure 11: PPO evaluation reward over training for `gait_v23`. Reward shaping improved performance relative to early survival-only behaviors, but PPO remained sensitive to reward design and did not sustain a stable gait.

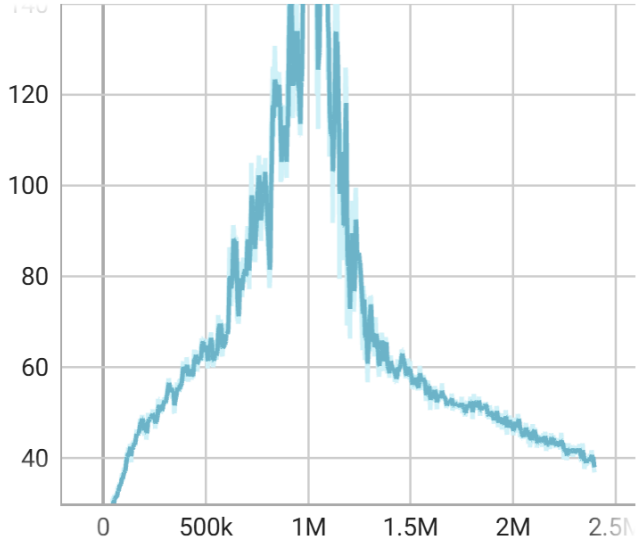


Figure 12: PPO rollout episode length during training. PPO frequently showed short periods of improvement followed by plateau or degradation, motivating repeated reward-shaping iterations.

are visible in Figs. 1, 3, and 5. We suspect the cause is the replay buffer slowly ageing out the “early-fall” transitions: once the buffer contains almost only successful walking trajectories, a small policy update can move into a region the critic has not seen in a long time, and the policy chases a stale gradient briefly before the entropy controller pushes exploration back up. Periodically mixing in older

Table 1: Final PPO vs. SAC comparison.

	PPO	SAC
On / off policy	on	off
Replay buffer	–	1 M
Steps trained	~1.4 M	~2.8 M
Best eval return	~3,000	~7,100
Best fwd speed	~0.4 m/s	~1.6 m/s
Throughput	~300 sps	~50 sps

transitions or using a smaller learning rate late in training would likely fix this.

Unconventional gaits. The robot did not converge to a clean human-style heel-toe gait. The final policy uses what looks more like a duck walk: knees stay bent, ankles push off in a short hop, and the torso leans slightly forward. This is a legitimate local optimum of the reward—it scores well on velocity, survival, and energy. Without a periodic phase reward or a motion-capture imitation term [6], there is no signal encouraging a particular limb-coordination pattern.

Locked arms. Holding the arms at zero simplifies training but also removes a useful degree of freedom for balance. Re-running with the full 21-joint action space is straightforward but would significantly change the credit-assignment problem.

Sim-to-real gap. We never deploy on the physical T1, so all numbers are simulation-only. Domain randomization (motor friction, link mass, latency) would be a natural next step before any hardware test.

4.7 Ethical considerations

The system here is a small humanoid in a single, flat simulation environment; the ethical scope is correspondingly narrow. Two issues are nonetheless worth flagging. First, learned locomotion controllers can encode unsafe behaviors that are hard to detect from training metrics alone (for instance, a policy that scores well on forward velocity but slams its feet into the ground hard enough to damage real hardware). Standard deployment hygiene—torque/velocity limits, hardware estops, and behavior-clone fallbacks—is sufficient to mitigate this in a research setting. Second, robots that walk reliably are dual-use; we are studying an academic platform but the same techniques are common in surveillance and military contexts. We do not view our specific contribution as moving that frontier, but flag it for completeness.

5 CONCLUSION AND FUTURE WORK

We trained a 13-DoF bipedal humanoid to walk in PyBullet using two standard deep-RL algorithms and a hand-shaped reward. SAC produced a stable forward gait at ~1.6 m/s after ~2.8 M environment steps; PPO under the same wall-clock budget plateaued at less than half of SAC’s return. The biggest practical lesson from the project is how much of the work is in the reward function rather than the algorithm: small changes to the survival bonus or the energy weight produced very different gaits, and most of the time we spent debugging was actually time spent diagnosing reward hacks.

If we had another month we would: (1) add domain randomization and a sim-to-real test on the physical T1 platform; (2) reintroduce the arm joints into the action space, since arm motion is critical for human walking and may help SAC stabilize at higher speeds; (3) replace the dense forward-velocity reward with a target-velocity tracking reward so the policy can be commanded at run time; (4) add a periodic gait reward or short imitation reference to encourage symmetric stepping; and (5) parallelize rollouts (e.g. Isaac Gym [8]) so a full PPO sweep takes hours rather than days.

6 CONTRIBUTION STATEMENT AND PEER EVALUATION

All three authors contributed substantively to the project. A short summary of individual contributions is below; the team agrees that contributions were roughly equivalent in effort, with the breakdown reflecting areas of focus rather than relative workload.

Drew Burcher Set up the PyBullet/Gymnasium environment and the Stable-Baselines3 training harness, including the auto-commit/metadata pipeline, the live training dashboard, and the evaluation/GIF recorder. Ran and tuned the SAC experiments and produced all SAC plots used in Section 4.

Dan Torri Led the PPO experiments: implemented the multiple reward-weight variants, ran the long PPO sweep, and produced the PPO TensorBoard plots and qualitative analysis. Drafted the related-work section and the PPO/SAC comparison.

Nathan Wyatt Designed and iterated on the reward function (especially the energy, joint-limit, and z-velocity terms), implemented the reward ablation script, and contributed the methodology and ethical-considerations text. Coordinated meetings and slide preparation for the in-class presentation.

Peer evaluation. Each member rates the other two as fully meeting expectations on attendance, follow-through, and code/writing quality. There were no disputes during the project. We make no asymmetric peer-evaluation claims.

REFERENCES

- [1] Erwin Coumans and Yunfei Bai. 2016–2024. PyBullet, a Python module for physics simulation for games, robotics and machine learning. <http://pybullet.org>.
- [2] Farama Foundation. 2023. Gymnasium: A Standard API for Reinforcement Learning Environments. <https://gymnasium.farama.org>.
- [3] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*.
- [4] Shuuji Kajita, Fumio Kanehiro, Kenji Kaneko, Kiyoshi Fujiwara, Kensuke Harada, Kazuhito Yokoi, and Hirohisa Hirukawa. 2003. Biped Walking Pattern Generation by Using Preview Control of Zero-Moment Point. In *IEEE International Conference on Robotics and Automation (ICRA)*.
- [5] Jonas Koenemann, Andrea Del Prete, Yuval Tassa, Emanuel Todorov, Olivier Stasse, Maren Bennewitz, and Nicolas Mansard. 2015. Whole-body Model-Predictive Control Applied to the HRP-2 Humanoid. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- [6] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. 2018. DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills. *ACM Transactions on Graphics (TOG)* 37, 4 (2018).
- [7] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22, 268 (2021), 1–8.
- [8] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. 2022. Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning. In *Conference on Robot Learning (CoRL)*.
- [9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [10] Jonah Siekmann, Yesh Godse, Alan Fern, and Jonathan Hurst. 2021. Sim-to-Real Learning of All Common Bipedal Gaits via Periodic Reward Composition. In *IEEE International Conference on Robotics and Automation (ICRA)*.

A HYPERPARAMETERS

Table 2 lists the algorithm hyperparameters used for the runs reported in Section 4. All values match the `config.py` file in the project repository.

Table 2: SAC and PPO hyperparameters used in the reported runs.

	SAC	PPO
Optimizer	Adam	Adam
Learning rate	3×10^{-4}	3×10^{-4}
Discount γ	0.99	0.99
Network	MLP [256, 256]	MLP [256, 256]
Batch size	256	64
Replay buffer	10^6	—
Soft update τ	0.005	—
Target entropy	−5 (auto α)	—
Rollout length	—	2,048
Epochs / update	—	10
GAE λ	—	0.95
Clip range	—	0.2
Entropy coef	—	0.01
Value coef	—	0.5

B REWARD WEIGHTS

Table 3 lists the per-term reward weights used for the SAC run reported in Section 4. The reward formula is given in Section 3.

Table 3: Reward weights for the reported SAC run.

Term	Weight
Forward velocity	+3.0
Survival (alive)	+2.0
Energy (mean power)	−0.00495
Fall (terminal)	−100.0
Orientation (roll/pitch sq.)	−0.5
Joint-limit (excess sq.)	−0.2
Z-velocity (downward)	−0.5

C AI USAGE STATEMENT

AI Tools were used including Claude (Anthropic), Gemini (Google), and the Overleaf AI for help with formatting and proofreading this document. AI was also used to help develop the code for this project.